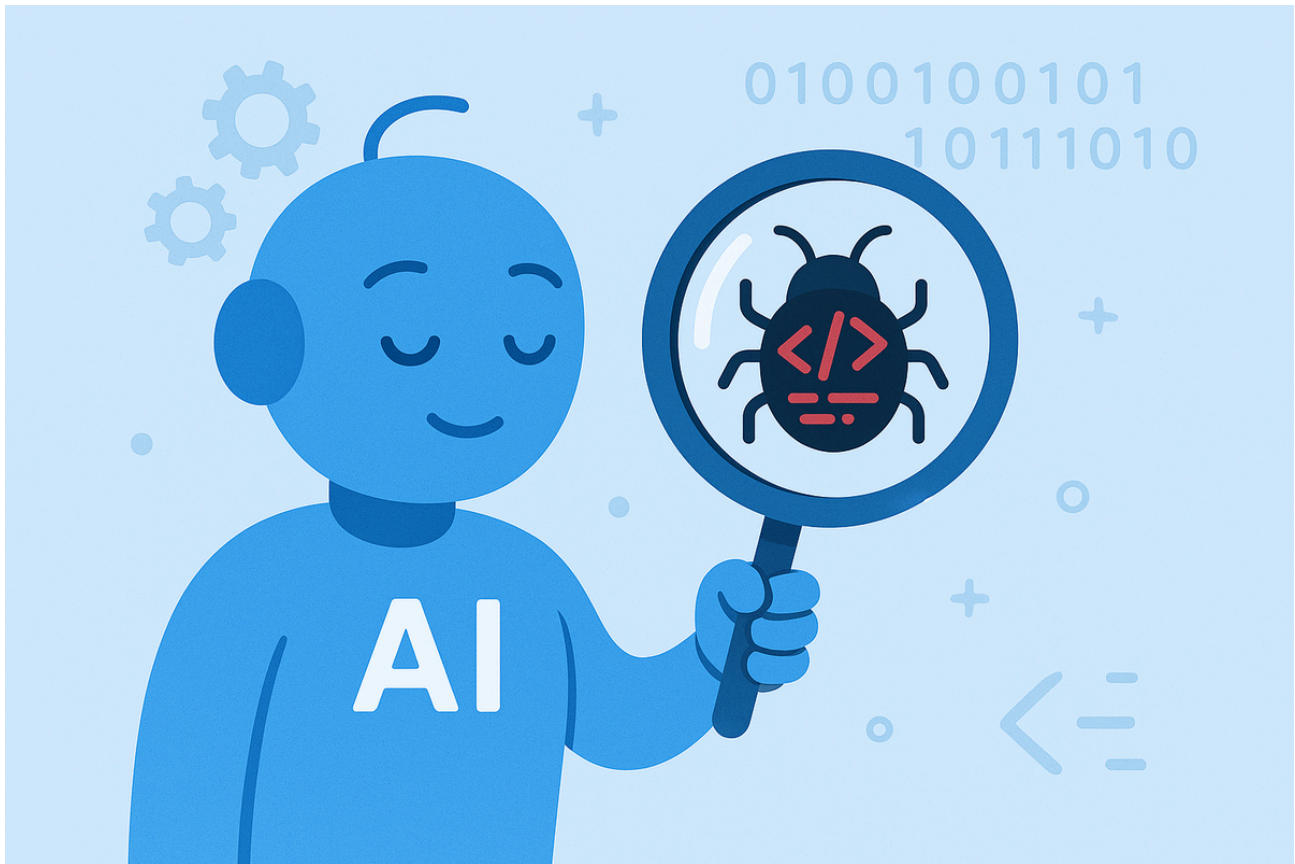


Automate Malware Analysis with AI

 minder-security.ghost.io/automate-malware-analysis-with-ai

Richard Minder

June 4, 2025



Analyzing Malware is a difficult task, which requires skill and a deep understanding of operating systems, executable file formats, process structures & technologies, and low-level programming languages such as C or Assembly. Additionally, the malware analyst needs to be familiar with common techniques (obfuscation, encryption/encoding, shellcode loading, process injection, etc.).

All in all, malware analysis can be a tough job, especially when the developers put some effort into packing and obfuscating the code.

In this blog post, I would like to explore GhidraMCP, which is a project that bridges Ghidra and LLMs using MCP (Model Context Protocol). MCP is a protocol designed for LLMs to **interface with local tools** securely and contextually. MCP is **still experimental and evolving**.

Note I am using the Claude App installed on my Windows 11 machine, but you can use any LLM which offers MCP functionality

Setup

MCP is based on a client-server structure, which means that we need a client (Ghidra) and a Server (GhidraMCP communicating with an LLM).

The following prerequisites must be met:

- Ghidra
- Python version 3.8 and above (I used 3.13)
I had to install the **requests** library via `pip install requests`
- Python MCP SDK (<https://github.com/modelcontextprotocol/python-sdk>)
I installed it using `pip install "mcp[cli]"`
- An LLM capable of executing local MCP requests
I used Claude with a Pro Subscription

1 - Install the Ghidra Extension#

GhidraMCP comes with a **Ghidra Extension** which we need to install to be able to interact with Ghidra:

1. Download the latest release from the GhidraMCP GitHub:
<https://github.com/LaurieWired/GhidraMCP/releases>
2. Unzip the file and store its contents somewhere useful
3. Run Ghidra
4. Select File -> Install Extensions
5. Click the + button
6. Select the ZIP file inside the folder of the extracted ZIP file
 1. If you get prompted due to a Version Mismatch, either ignore it or downgrade your Ghidra installation for max compatibility
7. Restart Ghidra
8. Make sure the GhidraMCPPlugin is enabled in File -> Configure -> Developer (needs to be checked)

2 - Configure Claude

Now that our Ghidra extension is installed and the MCP bridge is available, we need to tell Claude **where to find** said bridge (`bridge_mcp_ghidra.py`):

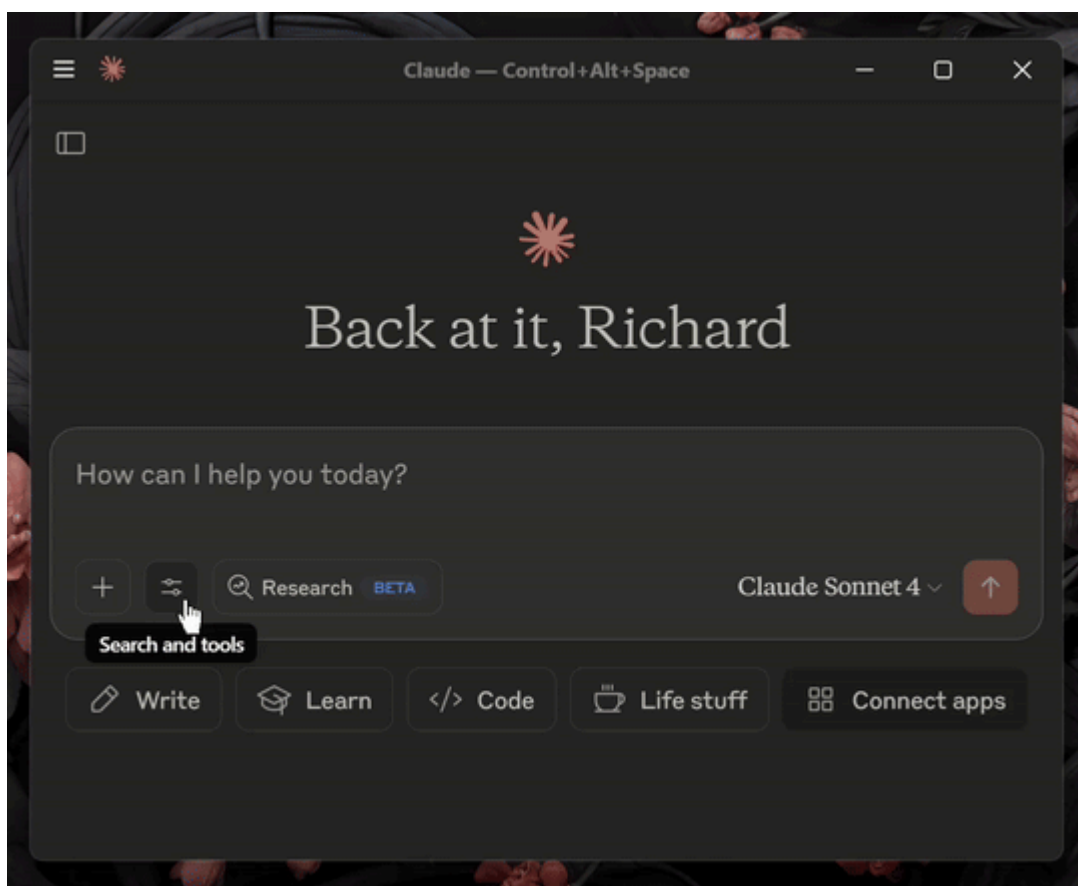
1. Open Claude Desktop
2. Go to **File > Settings**
3. Click on the **Developer** tab on the left
4. Select **Edit Config**. This will open your file explorer inside your Claude folder under `%APPDATA%`, where you can see `claude_desktop_config.json`

5. Open `claude_desktop_config.json` with your favorite editor and paste the following (replace the first argument with the path to your `bridge_mcp_ghidra.py` file):

```
{
  "mcpServers": {
    "ghidra": {
      "command": "python",
      "args": [
        "C:\\\\Tools\\\\GhidraMCP-release-1-3\\\\bridge_mcp_ghidra.py",
        "--ghidra-server",
        "http://127.0.0.1:8080/"
      ]
    }
  }
}
```

6. **Restart Claude** (make sure to fully quit the process, including from the system tray)

If everything worked (it might take a few restarts, the Claude App is a bit buggy at the moment), you should be able to see the Ghidra integration:



Info: How it works under the Hood#

We can see from the source code how this works:

1. Fast-MCP is used to create a MCP server called "ghidra-mcp". This handles connection management, protocol compliance, and message routing, basically exposing MCP tools and handling incoming requests from Claude:

```
from mcp.server.fastmcp import FastMCP
```

```
mcp = FastMCP("ghidra-mcp")
```

2. The individual functions available to the LLM are declared using `@mcp.tool()` decorators inside of `GhidraMCP/bridge_mcp_ghidra.py`. Let's look at `list_methods`, which lists the functions inside the currently open program:

```
@mcp.tool()
def list_methods(offset: int = 0, limit: int = 100) -> list:
    """
    List all function names in the program with pagination.
    """
    return safe_get("methods", {"offset": offset, "limit": limit})
```

3. The Ghidra Extension offers a HTTP server under `GhidraMCP/src/main/java/com/lauriewired/GhidraMCPPlugin.java` which handles the incoming request and forwards them to the appropriate handler function (in this case `getAllFunctionNames`):

```
server.createContext("/methods", exchange -> {
    Map<String, String> qparams = parseQueryParams(exchange);
    int offset = parseIntOrDefault(qparams.get("offset"), 0);
    int limit = parseIntOrDefault(qparams.get("limit"), 100);
    sendResponse(exchange, getAllFunctionNames(offset, limit));
});
```

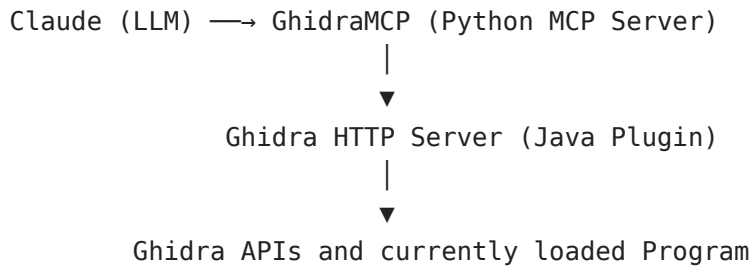
3. The handler functions are implemented within the same file, utilizing the Ghidra library to work with the currently open program:

```
private String getAllFunctionNames(int offset, int limit) {
    Program program = getCurrentProgram();
    if (program == null) return "No program loaded";

    List<String> names = new ArrayList<>();
    for (Function f : program.getFunctionManager().getFunctions(true)) {
        names.add(f.getName());
    }
    return paginateList(names, offset, limit);
}
```

4. The results are returned to the MCP Bridge, which are being returned to the LLM.

This is a diagram of the communication flow:



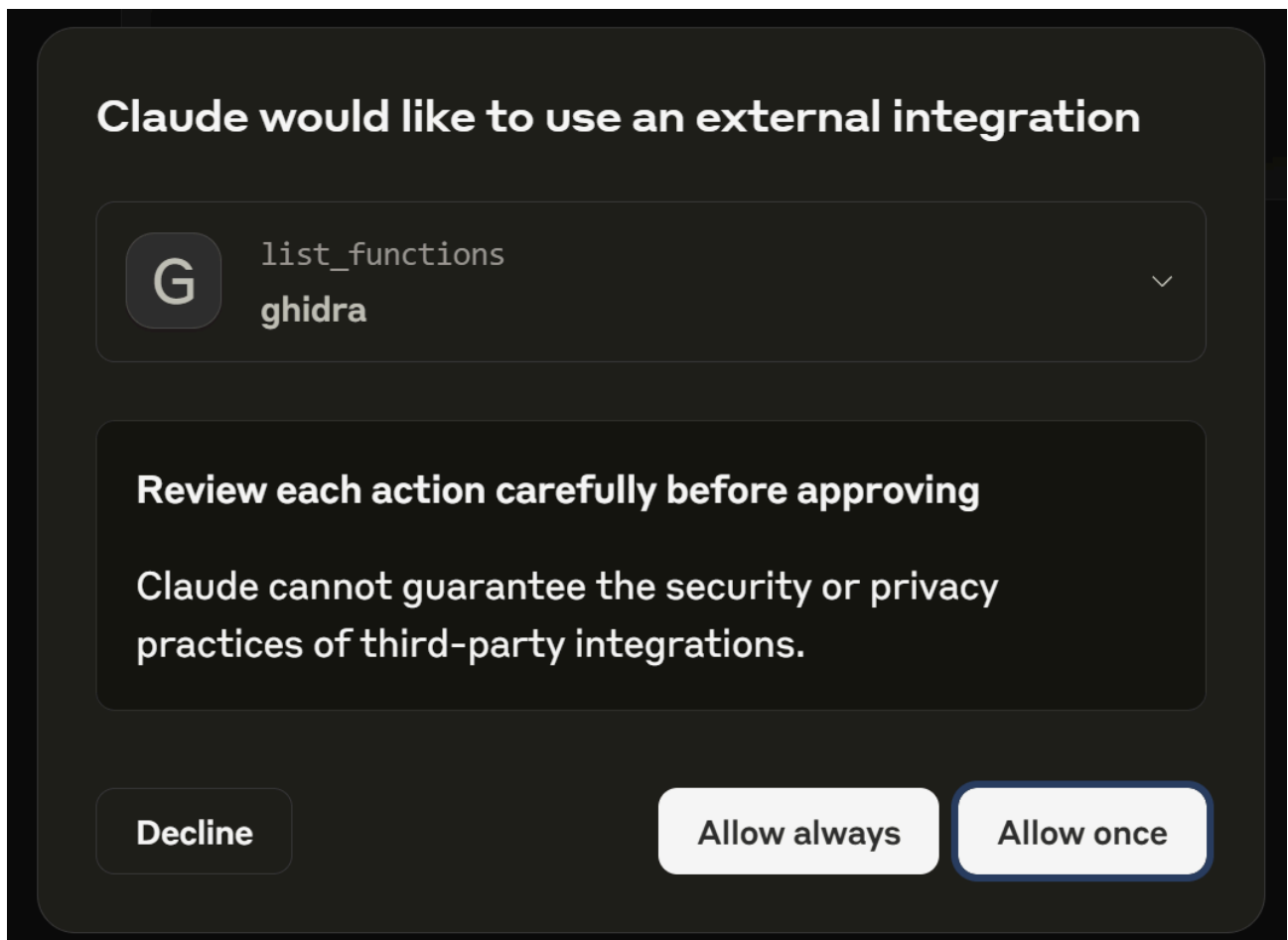
Reversing with Claude

Now that Claude can communicate with Ghidra. I chose to analyze a compiled version of an old Project of mine which demonstrated various process injection techniques.

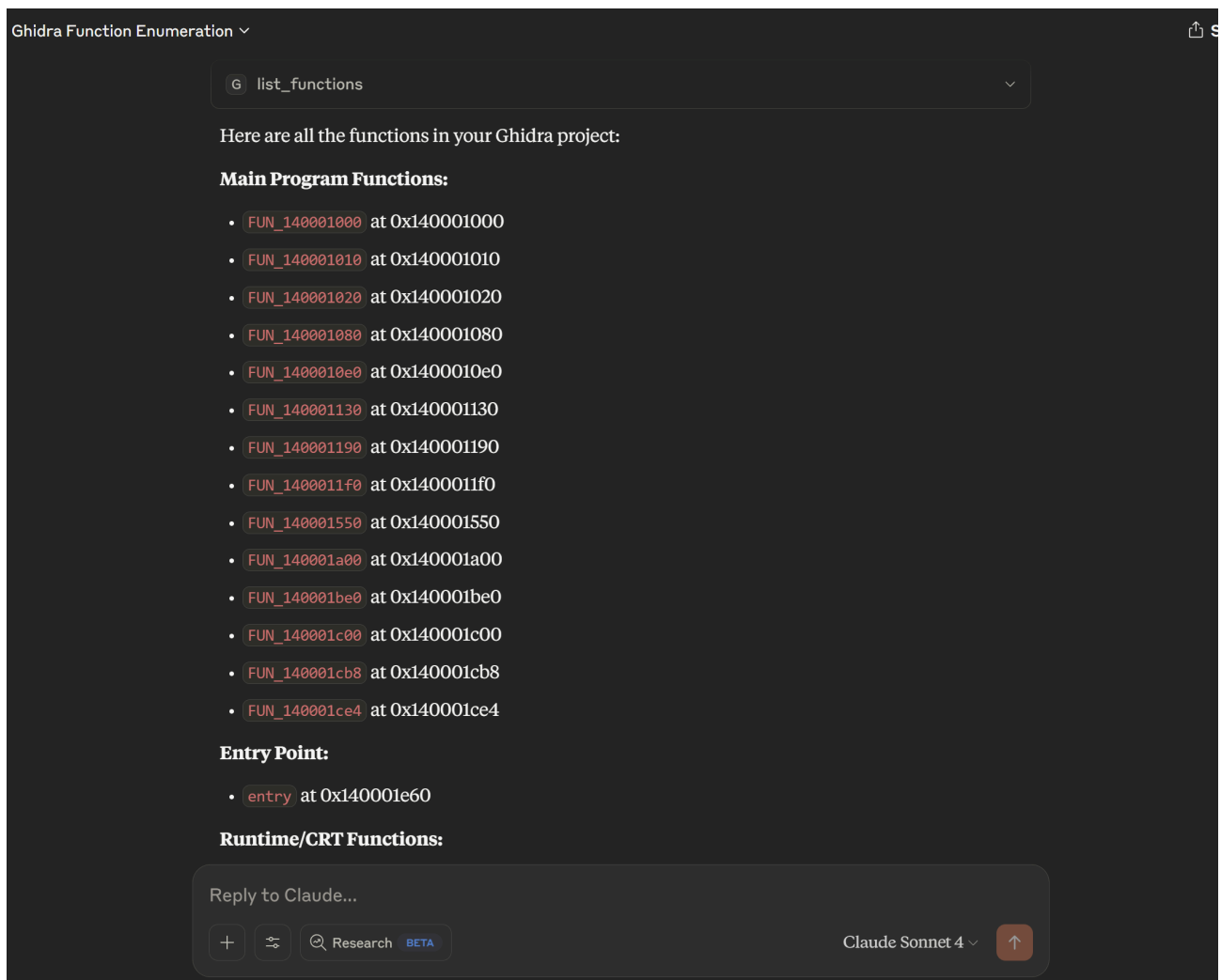
1. Create a new Project in Ghidra, and import your file
2. If prompted to configure new extensions, select "Yes" and check GhidraMCP
3. Then open Claude

Enumerating Functions#

I asked Claude to enumerate functions on my open Ghidra Project. Claude prompted me, asking me for permission:



I clicked "Allow always" and Claude went ahead and gave me a summary:



Renaming Functions#

I then asked Claude to rename the functions within my project. That was one of the features showcased by [LaurieWired](#) which I was especially excited for:

"Please rename all the main program functions with each function name being descriptive of the functionality provided by the function".

Claude then proceeded to first call `decompile_function` to understand the functionality of each function, and then called `rename_function` to rename them accordingly. During this, I could watch the changes being performed in my open Ghidra Disassembler, which I found fun.

Before:

```

FUN_140001130("\n\nEnter a valid PID, enter 0 to exit: ",puVar3,puVar6,param_4);
puVar4 = &local_268;
FUN_140001190(&DAT_140003dd0,puVar4,puVar6,param_4);
if (local_268 != 0) {
    FUN_140001130("\n----- STEP 2 - SELECTING AN INJECTION TECHNIQUE -----\n\n[1] - Dynam
ic-link Library Injection (T1055.001)\n[2] - Portable Executable Injection (T1055.002)\n[3] - Thr
ead Execution Hijacking (T1055.003)\n[4] - Asynchronous Procedure Call (T1055.004)\n"
        ,puVar4,puVar6,param_4);
    FUN_140001130("\nSelect a technique: ",puVar4,puVar6,param_4);
    piVar5 = local_264;
    FUN_140001190(&DAT_140003dd0,piVar5,puVar6,param_4);
    if (local_264[0] == 1) {
        FUN_1400011f0((ulonglong)local_268,piVar5,puVar6,param_4);
        goto LAB_140001b90;
    }
    if (local_264[0] == 2) {
        FUN_140001550((ulonglong)local_268,piVar5,puVar6,param_4);
        goto LAB_140001b90;
    }
}

```

After:

```

printf_wrapper("\n\nEnter a valid PID, enter 0 to exit: ",puVar3,puVar6,param_4);
puVar4 = &local_268;
scanf_wrapper(&DAT_140003dd0,puVar4,puVar6,param_4);
if (local_268 != 0) {
    printf_wrapper("\n----- STEP 2 - SELECTING AN INJECTION TECHNIQUE -----\n\n[1] - Dynam
ic-link Library Injection (T1055.001)\n[2] - Portable Executable Injection (T1055.002)\n[3] - Th
read Execution Hijacking (T1055.003)\n[4] - Asynchronous Procedure Call (T1055.004)\n"
        ,puVar4,puVar6,param_4);
    printf_wrapper("\nSelect a technique: ",puVar4,puVar6,param_4);
    piVar5 = local_264;
    scanf_wrapper(&DAT_140003dd0,piVar5,puVar6,param_4);
    if (local_264[0] == 1) {
        dll_injection_technique((ulonglong)local_268,piVar5,puVar6,param_4);
        goto LAB_140001b90;
    }
    if (local_264[0] == 2) {
        shellcode_injection_technique((ulonglong)local_268,piVar5,puVar6,param_4);
        goto LAB_140001b90;
    }
}

```

(Since this is a PE, I should have specified to follow the MSVC naming convention, but I'm also happy with underlines)

That was already very impressive, but now I wanted to see how far I could push Claude...

Renaming Functions, Variables, and Data#

I set up another prompt:

```

"Rename all functions and variables. Make the names descriptive enough for a
Malware Analyst to understand at a glance what they do"

```

- It renamed ALL functions, not just the ones I wrote. It did not name the actual main function `main()` but instead `main_menu_and_process_selection()`, which was good enough for me
- It then renamed a **large potion** of the variables. The ones which were "imagined" by the decompiler were left out, which did not disturb me. The final result was descriptive enough and would have already saved me some time:

```
kernel32_handle = GetModuleHandleW(L"kernel32.dll");
loadlibraryw_address =
(LPTHREAD_START_ROUTINE)GetProcAddress(kernel32_handle, "LoadLibraryW");
```

```
target_process_handle = OpenProcess(0x1fffff, 0, (DWORD)target_process_id);
```

```
remote_memory_address = VirtualAllocEx(target_process_handle,
(LPVOID)0x0, uVar8, 0x3000, 4);
```

```
remote_thread_handle =
    CreateRemoteThread(target_process_handle, (LPSECURITY_ATTRIBUTES)0x0, 0,
        loadlibraryw_address, remote_memory_address, 0,
(LPDWORD)0x0);
```

Much to my pleasant surprise, Claude then also proceeded to provide me with a summary document, which was also **very well written and structured**:

⚠ **BEHAVIORAL PATTERNS**

- **Process Discovery** (T1057) - Enumerates running processes
- **Process Injection** (T1055) - Multiple sub-techniques implemented
- **Obfuscated Files** (T1027) - XOR decryption capability
- **Defense Evasion** - PE validation, CPU feature checks

🔍 **DETECTION OPPORTUNITIES**

- Process enumeration API calls
- VirtualAllocEx with RWX permissions
- WriteProcessMemory + CreateRemoteThread combinations
- LoadLibraryW resolution and injection
- XOR decryption loops in memory

Verdict

I was very impressed with Claude's analysis skills and even more surprised when Claude provided me with a comprehensive summary. I see **enormous potential** in this technology and cannot wait for future updates.

I did not test deep deobfuscation skills yet, but I already used ChatGPT excessively to deobfuscate code in the past (through copy-paste), so I can expect Claude to perform well enough to save some time.

The future is exciting!

Thank you for reading my post!